Package ‘rENM.ai’

May 24, 2026

Type Package

Title GenAI tools for the rENM Framework

Version 0.1.0.9000

Description Provides generative AI integration tools for the rENM
Framework, including assembly of AI-ready suitability packages,
submission to large language model APIs (Claude and ChatGPT), and
rendering of returned interpretive documents.

Depends R (>= 4.1.0)

License MIT + file LICENSE

Encoding UTF-8

Roxygen list(markdown = TRUE)

URL `https://github.com/rENM-Framework/rENM.ai`

BugReports `https://github.com/rENM-Framework/rENM.ai/issues`

Imports rENM.core,
curl,
httr2,
utils

Config/roxygen2/version 8.0.0

Contents

assemble_ai_package *Assemble a GenAI suitability package for a species*

Description

Collects selected suitability-trend rasters, summary tables, and species metadata for a single species, then builds an AI-ready package together with prompt templates and a log entry. The function locates the project root via `rENM_project_dir()` and does not rely on hard-coded paths.

Usage

```
assemble_ai_package(
  alpha_code,
  files = c("centroids/Bioclimatic-Velocity.csv",
    "centroids/Centroids-Latitude-Summary.csv",
    "centroids/Centroids-Longitude-Summary.csv",
    "suitability/Suitability-Change-Trend.tif",
    "suitability/Suitability-Trend-State-Analysis-Summary.csv",
    "suitability/Suitability-Trend.tif", "suitability/Species-Information.csv",
    "variables/Variable-Contributions-BR-Stats.csv")
)
```

Arguments

`alpha_code` Character. Four-letter species alpha code (e.g., `"CASP"`).

`files` Character vector. File paths (without alpha_code prefix) identifying which outputs to include in the AI package. All entries must include a subdirectory relative to `Trends/`, for example: `"suitability/Suitability-Trend.tif"` or `"centroids/Bioclimatic-Velocity.csv"`.

Each file will be resolved as:

```
<project_root>/runs/<alpha_code>/Trends/<subdir>/<alpha_code>-<filename>
```

Defaults to a minimal core set.

Details

Pipeline context

- Copies selected suitability rasters and tables into a shared `suitability_package/` staging directory.
- Creates a `.zip` archive of the package contents.
- Copies the contents of `suitability_package/` and the corresponding prompt template into *two* provider subdirectories under `Summaries/`: `chatgpt/` (prompt from `inst/chatgpt/`) and `claude/` (prompt from `inst/claude/`).
- Copies `\_species.csv` to a species-specific file and includes it in the package.
- Appends a standardized processing-summary block to `\_log.txt`.

File selection

- Adds flexible file selection via `files`.

- All files are drawn from subdirectories under `Trends/`.
- Users must explicitly specify subdirectories (e.g., `"suitability/Suitability-Trend.tif"` or `"centroids/Bioclimatic-Velocity.csv"`).

Directory layout produced

```
runs/<alpha_code>/Summaries/
  chatgpt/
    suitability_package/   <- staged files
    suitability_package.zip
    suitability_prompt.txt <- from inst/chatgpt/
  claude/
    suitability_package/   <- same staged files
    suitability_package.zip
    suitability_prompt.txt <- from inst/claude/
```

Value

Named list (invisible) with components:

- `alpha_code` – input species code
- `pkg_dir` – path to staging `suitability_package/`
- `chatgpt_dir` – path to species `chatgpt/` folder
- `claude_dir` – path to species `claude/` folder
- `chatgpt_zip_path` – path to ChatGPT zip archive
- `claude_zip_path` – path to Claude zip archive
- `chatgpt_prompt_path` – path to copied ChatGPT prompt file
- `claude_prompt_path` – path to copied Claude prompt file
- `species_info_trends` – trends-level species table
- `species_info_pkg` – species table inside the staging package
- `log_file` – updated log file path

Examples

```
## Not run:
assemble_ai_package("CASP")

## End(Not run)
```

`render_ai_docx` *Render GenAI DOCX report to PDF*

Description

Converts a species-specific DOCX report into PDF and appends a processing summary to the run log.

Usage

```
render_ai_docx(alpha_code, verbose = TRUE)
```

Arguments

alpha_code	Character. Species alpha code, for example "CASP".
verbose	Logical. If TRUE (default), prints diagnostic messages.

Details

Inputs

- One DOCX file per species located at <rENM_project_dir()>/runs/<alpha_code>/Summaries/pages/ <alpha_code>-Suitability-Trend-Analysis.docx

Outputs

- A PDF written to the same directory: <alpha_code>-Suitability-Trend-Analysis.pdf
- A multi-line processing summary appended to <rENM_project_dir()>/runs/<alpha_code>/_log.txt

Methods

- Conversion uses LibreOffice soffice in headless mode.
- File sizes and time stamps are recorded for auditing.

Value

Character. Invisibly returns the path to the generated PDF.
Side effects:

- Writes a PDF file alongside the source DOCX.
- Appends a formatted record to the species-level _log.txt.

Examples

```
## Not run:
# Convert the Cassin's Sparrow report and view the resulting PDF path
pdf_path <- render_ai_docx("CASP")

## End(Not run)
```

`submit_to_chatgpt` *Submit a GenAI-ready climatic suitability trend package to ChatGPT*

Description

Interprets analysis outputs from an rENM run for a single species by sending a zipped data bundle and custom prompt to the OpenAI Responses API, retrieving a DOCX report, and logging processing metadata and cost information.

Usage

```
submit_to_chatgpt(
  alpha_code,
  model = "gpt-5.1",
  api_key = Sys.getenv("OPENAI_API_KEY")
)
```

Arguments

`alpha_code`	Character. Four-letter species alpha code.
`model`	Character. OpenAI model identifier.
`api_key`	Character. OpenAI API key.

Details

Pipeline context

- Reads a species-specific prompt file.
- Uploads a zipped bundle of GeoTIFF and CSV inputs.
- Calls the Responses API with the code-interpreter tool enabled.
- Downloads a Word report created inside the container.
- Computes token usage and approximate OpenAI cost.
- Appends an eBird-style summary line to `_log.txt`.

Expected inputs

- Prompt file containing the placeholder string `<alpha_code>`.
- Zipped bundle of raster and tabular data required by the prompt.

Authentication

- Uses `api_key`, defaulting to `Sys.getenv("OPENAI_API_KEY")`.

Directory layout under `rENM_project_dir()`

```
<project_dir>/runs/CASP/Summaries/chatgpt/suitability_package.zip
<project_dir>/runs/CASP/Summaries/chatgpt/suitability_prompt.txt
<project_dir>/runs/<alpha_code>/Summaries/pages/
    <alpha_code>-Suitability-Trend-Analysis.docx
<project_dir>/runs/<alpha_code>/_log.txt
```

Outputs

- DOCX report saved to the pages directory.
- Updated `_log.txt` containing a processing summary.

Value

List (invisible) with components

- `response`
- `docx_path`
- `elapsed_sec`
- `input_tokens`, `output_tokens`, `total_tokens`
- `est_total_cost`

`submit_to_claude` *Submit a GenAI-ready climatic suitability trend package to Claude*

Description

Interprets analysis outputs from an rENM run for a single species by uploading a zipped data bundle to the Anthropic Files API, referencing it as a `file` content block in the Messages API call with the code execution tool enabled, retrieving the generated DOCX report, and logging processing metadata and cost information.

Usage

```
submit_to_claude(
  alpha_code,
  model = "claude-sonnet-4-6",
  api_key = Sys.getenv("ANTHROPIC_API_KEY"),
  max_tokens = 32000L,
  timeout_sec = 600L,
  delete_input_file = TRUE
)
```

Arguments

`alpha_code`	Character. Four-letter species alpha code (e.g., `"CASP"`).
`model`	Character. Anthropic model identifier. Defaults to `"claude-sonnet-4-6"`.
`api_key`	Character. Anthropic API key. Defaults to the value of the `ANTHROPIC_API_KEY` environment variable.
`max_tokens`	Integer. Maximum output tokens. Defaults to 32000. The code execution tool consumes tokens across multiple internal round-trips (code submission, stdout, file output), so this task requires significantly more headroom than a plain text generation call. A `stop_reason` of `"max_tokens"` in the response indicates this limit was reached before the DOCX was produced; increase to 64000 if needed.
`timeout_sec`	Integer. Request timeout in seconds. Defaults to 600 (10 minutes). Increase for very large packages.
`delete_input_file`	Logical. If `TRUE` (default), the uploaded zip is deleted from Anthropic storage after the run.

Details

What is sent to the API

1. `suitability_package.zip` is uploaded to the Anthropic Files API, returning a `file_id`.
2. `suitability_prompt.txt` is read and `<alpha_code>` is interpolated.
3. The Messages API is called with two content blocks: a `text` block containing the prompt, and a `container_upload` block referencing the uploaded zip by `file_id`. The `container_upload` block type is the correct mechanism for delivering binary files to the code execution sandbox via the Files API.
4. The uploaded zip is deleted from Anthropic storage after the run to avoid accumulating storage usage.

Why not base64 embedding?

Embedding the zip as base64 in the prompt body produces a ~74,000 character payload that causes the Anthropic API to time out before responding (zero bytes received). Uploading via the Files API and referencing by `file_id` keeps the Messages API request small and avoids this problem.

Pipeline context

- Uploads zip to Files API; records `file_id`.
- Calls Messages API with `text` + `file` content blocks and the code execution tool enabled (beta: `code-execution-2025-08-25`).
- Saves the raw API response to `debug_resp.rds` immediately after the call, regardless of outcome.
- Scans response blocks for output file IDs and downloads the DOCX via the Files API (`files-api-2025-04-14`).
- Deletes the uploaded input zip from Anthropic storage.
- Estimates token usage and approximate API cost.
- Appends a processing summary to `_log.txt`.

Debugging a failed run

If no DOCX is produced, inspect the saved response:

```
resp <- readRDS("<species_dir>/debug_resp.rds")
submit_to_claude_diag(resp)
```

`submit_to_claude_diag()` prints block types, stdout/stderr from the sandbox, return codes, and any text Claude returned.

Authentication

- Uses `api_key`, defaulting to `Sys.getenv("ANTHROPIC_API_KEY")`.
- Obtain an API key from `https://console.anthropic.com`.
- This requires a separate Anthropic API account billed per token, independently of any claude.ai subscription.

Required beta features

Both are enabled automatically via the `anthropic-beta` header:

- `files-api-2025-04-14` – file upload and download
- `code-execution-2025-08-25` – sandboxed Python execution

Directory layout under `rENM_project_dir()`

```
<project_dir>/runs/<alpha_code>/Summaries/claude/suitability_package.zip
<project_dir>/runs/<alpha_code>/Summaries/claude/suitability_prompt.txt
<project_dir>/runs/<alpha_code>/Summaries/pages/
    <alpha_code>-Suitability-Trend-Analysis.docx
<project_dir>/runs/<alpha_code>/debug_resp.rds  <- saved after every run
<project_dir>/runs/<alpha_code>/_log.txt
```

Cost estimation

Approximate costs using claude-sonnet-4-6 pricing as of May 2026:

- Input tokens: $3.00 / 1M tokens
- Output tokens: $15.00 / 1M tokens

These are estimates only; actual billing may differ.

Value

Named list with components:

- `docx_path` – local path to the downloaded DOCX, or `NA` if not found.
- `elapsed_sec` – wall-clock seconds for the API call.
- `input_tokens` – input token count from usage metadata.
- `output_tokens` – output token count from usage metadata.
- `total_tokens` – total token count.
- `est_cost_usd` – estimated cost in USD.
- `response` – full parsed response body (list).
- `debug_rds` – path to the saved debug_resp.rds file.

Examples

```
## Not run:
  Sys.setenv(ANTHROPIC_API_KEY = "sk-ant-...")

  result <- submit_to_claude("CASP")
  message("Report: ", result$docx_path)
  message(sprintf("Cost: $%.6f | Tokens: %d",
                  result$est_cost_usd, result$total_tokens))

  # If no DOCX was produced, diagnose:
  submit_to_claude_diag(result$response)
  # or load from disk:
  submit_to_claude_diag(readRDS(result$debug_rds))

## End(Not run)
```

submit_to_claude_diag *Diagnose a failed submit_to_claude() response*

Description

Prints a structured summary of the raw API response from submit_to_claude to help identify why no DOCX was produced. Shows block types, sandbox stdout/stderr, return codes, and any text Claude returned.

Usage

```
submit_to_claude_diag(resp)
```

Arguments

resp	List. The parsed API response, either from result$response or readRDS("<species_dir>/debug_

Value

NULL invisibly (called for side effects).

Examples

```
## Not run:
  submit_to_claude_diag(result$response)
  submit_to_claude_diag(readRDS("runs/CASP/debug_resp.rds"))

## End(Not run)
```

Index